

PROJECT REPORT ON

CREDIT CARD APPROVAL PREDICTION SYSTEM

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Department of Artificial Intelligence and Data Science
Academic Year 2026 – 2027

Team Members

This project was carried out collaboratively by the following team members, whose individual contributions spanned data preprocessing, exploratory analysis, model development, and the design and implementation of the Flask web application.

- **Moram Rashmi Sri**

Team Lead

- **Akula Chandra Sekhar**

Member

- **Chandrika Battula**

Member

- **Anusha Kadaru**

Member

- **Nakka Sruthi**

Member

Abstract

Access to credit is a cornerstone of modern personal and household financial planning, and the decision to extend a credit card to an applicant carries significant risk for financial institutions. Traditional credit approval processes have historically relied on manual underwriting and rule-based scoring systems, which are often slow, inconsistent, and susceptible to human bias. This project, titled “Credit Card Approval Prediction System using Machine Learning,” addresses these limitations by developing a data-driven, automated framework capable of predicting the creditworthiness of an applicant based on demographic, financial, and credit-history attributes.

The system is built using two publicly available datasets sourced from Kaggle — `application_record.csv`, which captures applicant demographic and financial details, and `credit_record.csv`, which records monthly repayment behaviour. These datasets are merged, cleaned, and transformed through a structured preprocessing pipeline that includes duplicate removal, missing-value imputation, categorical encoding, and derivation of a binary target label from the `STATUS` field. Extensive exploratory data analysis is performed to understand the underlying distribution of applicant attributes such as income, occupation, education, and family composition, and to identify relationships between these attributes and credit risk.

Three supervised classification algorithms — Logistic Regression, Decision Tree, and Random Forest — are trained and evaluated on the processed dataset using standard performance metrics including accuracy, precision, recall, and F1-score. Comparative analysis shows that the Random Forest classifier consistently outperforms the other models in terms of predictive stability and generalisation, and it is therefore selected as the deployment model. The trained model is serialized using Joblib and integrated into a Flask-based web application that allows an end user to submit applicant details through a responsive HTML form and instantly receive a prediction regarding credit card approval. The project demonstrates a complete, end-to-end machine learning workflow, from raw data ingestion to a functioning, deployable prediction system, and lays the foundation for future enhancements such as explainable AI, cloud deployment, and real-time scoring APIs.

Table of Contents

Team Members	2
Abstract.....	3
Table of Contents.....	4
Chapter 1: Introduction	6
1.1 Background.....	7
1.2 Problem Statement.....	7
1.3 Objectives of the Project	7
1.4 Scope of the Project.....	8
1.5 Organisation of the Report	8
Chapter 2: Existing System and Literature Survey	8
2.1 Traditional Credit Approval Systems	8
2.2 Statistical Credit Scoring.....	8
2.3 Machine Learning-Based Credit Scoring.....	9
2.4 Gap Identified and Motivation for this Project	9
Chapter 3: Dataset Description	9
3.1 Data Source.....	9
3.2 application_record.csv.....	9
3.3 credit_record.csv	10
3.3.1 Understanding MONTHS_BALANCE	10
3.3.2 The STATUS Column and Target Variable Derivation	11
3.4 Necessity of Merging the Two Datasets.....	11
Chapter 4: Data Preprocessing.....	11
4.1 Reading and Inspecting the Datasets.....	11
4.2 Duplicate Removal	12
4.3 Missing Value Handling.....	12
4.4 Cleaning DAYS_BIRTH and DAYS_EMPLOYED	12
4.5 Categorical Encoding	12
4.6 Feature Engineering.....	13
4.6.1 Family Member Feature	13
4.6.2 Target Variable Construction	13
4.7 Merged Dataset Generation.....	13
4.8 Final Processed Dataset	13
Chapter 5: Exploratory Data Analysis	13

5.1 Descriptive Statistics	14
5.2 Univariate Analysis	14
5.2.1 Occupation Distribution	14
5.2.2 Income Distribution.....	14
5.2.3 Education Distribution.....	15
5.2.4 Family Status	15
5.2.5 Housing Type.....	15
5.3 Multivariate Analysis	15
5.3.1 Correlation Heatmap	15
5.3.2 Income versus Occupation	15
5.4 Summary of EDA Findings.....	16
Chapter 6: System Architecture	16
6.1 Pipeline Flow	16
6.2 Explanation of Each Block.....	16
Chapter 7: Machine Learning Methodology	17
7.1 Logistic Regression	17
7.1.1 Working Principle	17
7.1.2 Hyperparameters	17
7.1.3 Advantages and Disadvantages	18
7.2 Decision Tree.....	18
7.2.1 Working Principle	18
7.2.2 Hyperparameters	18
7.2.3 Advantages and Disadvantages	18
7.3 Random Forest	18
7.3.1 Working Principle	18
7.3.2 Hyperparameters	19
7.3.3 Advantages and Disadvantages	19
7.4 Training Process	19
7.5 Evaluation Metrics.....	19
Chapter 8: Model Results and Comparison	20
8.1 Comparative Performance.....	20
8.2 Model Comparison Table	21
8.2.1 Advantages and Disadvantages Recap	21
8.3 Random Forest — Confusion Matrix.....	21
8.4 Why Random Forest was Selected as the Deployment Model	22

Chapter 9: Web Application	23
9.1 Frontend	23
9.1.1 Prediction Page and Result Display	23
9.2 Backend	24
9.2.1 Flask Application Structure	24
9.2.2 Joblib Model Loading	24
9.2.3 Converting User Input into Model Features	24
9.2.4 Data Validation	24
9.2.5 Prediction Flow	24
9.3 Deployment Model	24
Chapter 10: Project Structure	25
Chapter 11: Screenshots	26
11.1 Dataset Preview	26
11.2 Exploratory Data Analysis	26
11.3 Model Training and Random Forest Results	26
11.4 Comparison Table	26
11.5 Flask Home Page, Prediction Page, and Prediction Result	26
11.6 Folder Structure	26
Chapter 12: Future Enhancements	26
1. Explainable AI (XAI) using SHAP	26
2. Full Cloud Deployment on IBM Cloud	27
3. User Authentication and Role-Based Access	27
4. Credit Score Prediction (Regression Extension)	27
5. Deep Learning-Based Models	27
6. Real-Time Prediction API	27
7. Model Monitoring and Drift Detection	27
8. Mobile Application	27
9. Database Integration	28
10. Automated Retraining Pipeline (MLOps)	28
Chapter 13: Conclusion	28
References	29

Chapter 1: Introduction

1.1 Background

Credit cards have become an integral part of modern financial ecosystems, enabling consumers to make purchases, build credit history, and manage short-term liquidity. However, the decision of whether to approve or reject a credit card application is a critical risk-management task for banks and financial institutions. An overly lenient approval policy exposes the institution to default risk and non-performing assets, while an overly conservative policy denies credit to genuinely eligible customers and results in lost business opportunities. Historically, this decision has been made through manual underwriting, where loan officers evaluate applications against a fixed set of eligibility rules. This process is time-consuming, inconsistent across officers, and difficult to scale as the volume of applications grows.

With the proliferation of digital banking and the availability of large volumes of historical applicant and repayment data, machine learning has emerged as a powerful alternative to rule-based credit scoring. Machine learning models can learn complex, non-linear relationships between an applicant's demographic and financial attributes and their eventual repayment behaviour, and can apply this learned knowledge consistently and instantaneously to new applications. This project explores the application of supervised machine learning techniques to the problem of credit card approval prediction, using real-world structured data obtained from Kaggle.

1.2 Problem Statement

Given an applicant's demographic information (such as age, gender, marital status, and family size), financial information (such as income, employment status, and asset ownership), and historical credit repayment behaviour, the objective of this project is to build a predictive model that can classify whether a new applicant is likely to be a good credit risk (approved) or a poor credit risk (rejected). The problem is formulated as a binary classification task, where the target label is derived from the applicant's historical repayment status recorded in the credit bureau data.

1.3 Objectives of the Project

- To collect, merge, and preprocess the `application_record` and `credit_record` datasets into a single, model-ready dataset.
- To perform detailed exploratory data analysis (EDA) to understand the distribution of applicant attributes and their relationship with credit risk.
- To engineer meaningful features from raw data fields, including age, years of employment, and family size.
- To train and evaluate multiple supervised classification algorithms — Logistic Regression, Decision Tree, and Random Forest — on the processed dataset.
- To compare the performance of these algorithms using standard classification metrics and select the best-performing model for deployment.

- To develop a Flask-based web application that allows end users to interactively submit applicant details and receive a real-time credit approval prediction.
- To design the system so that it can be extended to a cloud deployment environment such as IBM Cloud in future iterations.

1.4 Scope of the Project

The scope of this project is limited to the design, implementation, and evaluation of a supervised machine learning pipeline for binary credit approval classification, together with a lightweight web interface for demonstrating real-time predictions. The project does not implement live integration with an actual banking core system or credit bureau, and the datasets used are historical, anonymised, and sourced from a public repository rather than a production environment. Nonetheless, the pipeline, methodology, and web application architecture developed in this project are directly transferable to a production-grade credit risk assessment system with appropriate data governance and regulatory compliance measures.

1.5 Organisation of the Report

The remainder of this report is organised as follows. Chapter 2 reviews the existing approaches to credit scoring and situates this project within that context. Chapter 3 describes the dataset in detail. Chapter 4 explains the data preprocessing pipeline. Chapter 5 presents the exploratory data analysis. Chapter 6 describes the overall system architecture. Chapter 7 details the machine learning methodology and the algorithms used. Chapter 8 presents the model training results and comparative evaluation. Chapter 9 describes the Flask web application. Chapter 10 explains the project's folder structure. Chapter 11 presents representative screenshots of the system. Chapter 12 discusses future enhancements, and Chapter 13 concludes the report.

Chapter 2: Existing System and Literature Survey

2.1 Traditional Credit Approval Systems

Conventional credit approval systems rely predominantly on rule-based scorecards and manual review by credit officers. These systems typically apply fixed thresholds on income, existing debt obligations, and employment stability to arrive at an approval decision. While such systems are interpretable and easy to audit, they suffer from several drawbacks: they cannot easily capture non-linear interactions between variables, they require frequent manual recalibration as economic conditions change, and they tend to generalise poorly to applicant profiles that were not anticipated when the rules were designed.

2.2 Statistical Credit Scoring

Classical statistical techniques such as logistic regression and discriminant analysis have been used in the credit scoring industry for decades, most notably in the widely adopted FICO scoring methodology. These

approaches model the probability of default as a function of applicant attributes and provide a transparent, coefficient-based interpretation of risk factors. However, purely linear statistical models often underperform when the relationship between applicant attributes and default risk is highly non-linear or involves complex interactions between multiple variables.

2.3 Machine Learning-Based Credit Scoring

Recent research has demonstrated that ensemble-based machine learning algorithms, particularly tree-based methods such as Random Forest and Gradient Boosting, can significantly outperform traditional statistical models on credit scoring tasks by capturing complex feature interactions and being robust to noisy or irrelevant features. Machine learning approaches also allow institutions to continuously retrain models as new repayment data becomes available, keeping the scoring system aligned with current applicant behaviour and macroeconomic conditions.

2.4 Gap Identified and Motivation for this Project

While numerous academic studies have applied machine learning to credit scoring datasets, many of them use small, synthetic, or highly pre-processed benchmark datasets that do not reflect the noise and complexity of real applicant records. This project uses a large, realistic Kaggle dataset that closely mirrors the structure of actual bank application and credit bureau records, requiring substantial data cleaning, merging, and feature engineering before modelling can begin. This makes the project representative of the full end-to-end effort required to build a production-oriented credit risk model, rather than a simplified academic exercise, and additionally extends the work with a deployable web interface, which is often absent from purely research-oriented studies.

Chapter 3: Dataset Description

3.1 Data Source

The dataset used in this project is the “Credit Card Approval Prediction” dataset publicly available on Kaggle. It consists of two separate but related CSV files that together simulate the kind of data a bank would hold on its credit card applicants: `application_record.csv`, which contains demographic and socio-economic information collected at the time of application, and `credit_record.csv`, which contains the applicant's month-by-month credit repayment history over the period they have held an account.

3.2 `application_record.csv`

This file contains one row per applicant and captures the information typically gathered on a credit card application form. The key columns are described below.

Column	Description
ID	Unique identifier for each applicant; used as the join key with credit_record.csv
CODE_GENDER	Applicant's gender (M / F)
FLAG_OWN_CAR	Whether the applicant owns a car (Y / N)
FLAG_OWN_REALTY	Whether the applicant owns real estate / property (Y / N)
CNT_CHILDREN	Number of children of the applicant
AMT_INCOME_TOTAL	Total annual income of the applicant
NAME_INCOME_TYPE	Category of income source, e.g. Working, Commercial associate, Pensioner
NAME_EDUCATION_TYPE	Highest level of education attained
NAME_FAMILY_STATUS	Marital / family status, e.g. Married, Single, Civil marriage
NAME_HOUSING_TYPE	Type of accommodation, e.g. House / apartment, Rented apartment
DAYS_BIRTH	Applicant's age expressed as a negative count of days before the application date
DAYS_EMPLOYED	Number of days employed, expressed as a negative value; a large positive value denotes unemployment/pension
FLAG_MOBIL / FLAG_WORK_PHONE / FLAG_PHONE / FLAG_EMAIL	Binary indicators of contact information provided
OCCUPATION_TYPE	Applicant's occupation category, e.g. Laborers, Managers, Core staff
CNT_FAM_MEMBERS	Total number of family members in the applicant's household

3.3 credit_record.csv

This file contains the monthly credit history of each applicant, with one row per applicant per month. The key columns are described below.

Column	Description
ID	Applicant identifier, corresponding to the ID column in application_record.csv
MONTHS_BALANCE	The month of the record relative to the application month, expressed as 0, -1, -2, and so on, where 0 is the most recent month on record
STATUS	Repayment status for that month (see Section 3.4 for the coding scheme)

3.3.1 Understanding MONTHS_BALANCE

The MONTHS_BALANCE column allows the repayment history of every applicant to be reconstructed as a time series relative to the account opening date. A value of 0 represents the most recent recorded month,

-1 the month before that, and so forth. In this project, MONTHS_BALANCE is used primarily to identify the full repayment history available for each applicant before it is aggregated into a single risk label, ensuring that the model is trained on a comprehensive summary of an applicant's behaviour rather than a single, potentially unrepresentative month.

3.3.2 The STATUS Column and Target Variable Derivation

The STATUS column encodes the applicant's payment status for a given month using the following scheme: 0 indicates 1–29 days past due, 1 indicates 30–59 days past due, 2 indicates 60–89 days past due, 3 indicates 90–119 days past due, 4 indicates 120–149 days past due, 5 indicates a write-off or bad debt exceeding 150 days past due, C indicates that the account was paid off that month, and X indicates that there was no loan outstanding for that month.

To convert this granular, month-level status into a single binary target suitable for classification, every applicant's STATUS history is scanned, and the applicant is labelled as a high-risk / rejected case (target = 1) if any of their monthly records fall into the categories denoting 30 or more days past due (STATUS values 1 through 5). If none of an applicant's records indicate such delinquency — that is, all their records are C, X, or 0 — the applicant is labelled as a good credit risk / approved case (target = 0). This conservative, worst-case aggregation ensures that the resulting model is trained to identify applicants who have shown any history of serious delinquency, which is the behaviour most relevant to a bank's risk appetite.

3.4 Necessity of Merging the Two Datasets

Neither dataset is sufficient on its own to train a meaningful credit risk model. application_record.csv describes who the applicant is but contains no information about how they actually behaved once they held an account, while credit_record.csv describes repayment behaviour but contains no demographic or financial context that could explain or predict that behaviour. Merging the two datasets on the common ID column produces a single dataset in which every applicant's demographic and financial profile is paired with a derived label describing their actual repayment risk, which is precisely the supervised learning setup required to train a classifier that can predict risk for new applicants at the point of application, before any repayment history exists.

Chapter 4: Data Preprocessing

Raw application and credit record data cannot be fed directly into a machine learning algorithm; it must first be cleaned, transformed, and structured into a consistent numerical format. This chapter describes each preprocessing step applied in this project and explains why it was necessary.

4.1 Reading and Inspecting the Datasets

Both CSV files are loaded into Pandas DataFrames using `pandas.read_csv()`. Initial inspection is carried out using functions such as `.head()`, `.info()`, `.shape`, and `.describe()` to understand the number of records, the data types of each column, the presence of missing values, and the basic statistical spread of numerical fields. This step is essential because it establishes a baseline understanding of the data's quality and structure before any transformation is applied, and it often reveals data quality issues — such as unexpected data types or placeholder values — that must be addressed in subsequent steps.

4.2 Duplicate Removal

Duplicate rows, where they exist, are identified using the `.duplicated()` method and removed with `.drop_duplicates()`. Duplicate records can arise from data collection errors or repeated exports, and if left in the dataset they can bias the model by over-representing certain applicant profiles, artificially inflating validation performance through data leakage between the training and test sets.

4.3 Missing Value Handling

The `OCCUPATION_TYPE` column in `application_record.csv` contains a substantial number of missing values, corresponding to applicants who are unemployed, retired, or otherwise did not report an occupation. Rather than dropping these rows — which would discard a large portion of usable data — missing values in categorical columns are imputed with a placeholder category such as “Unknown,” preserving the row for training while explicitly flagging the absence of this information as a feature in its own right. Numerical columns are checked for missing values and, where present, are imputed using the column median, which is more robust to outliers than the mean.

4.4 Cleaning DAYS_BIRTH and DAYS_EMPLOYED

Both `DAYS_BIRTH` and `DAYS_EMPLOYED` are recorded as negative integers counting backwards from the application date, which is a common convention in this type of anonymised bureau data but is not directly interpretable. `DAYS_BIRTH` is converted to a positive `AGE_YEARS` feature by taking its absolute value and dividing by 365.25. `DAYS_EMPLOYED` contains a known data artefact in which unemployed or retired applicants are assigned a large positive placeholder value (365243), which is first replaced with 0 or treated as a separate “not currently employed” flag before the remaining negative values are converted to a positive `EMPLOYMENT_YEARS` feature in the same manner as age. This conversion is necessary because raw negative day-counts have no direct, intuitive relationship with credit risk, whereas age in years and years of employment are interpretable, monotonic features that align with how a human underwriter would reason about an applicant.

4.5 Categorical Encoding

Machine learning algorithms in Scikit-Learn require numerical input, so all categorical columns — including `CODE_GENDER`, `FLAG_OWN_CAR`, `FLAG_OWN_REALTY`, `NAME_INCOME_TYPE`,

NAME_EDUCATION_TYPE, NAME_FAMILY_STATUS, NAME_HOUSING_TYPE, and OCCUPATION_TYPE — are transformed using Label Encoding via Scikit-Learn's `LabelEncoder` class. Label Encoding assigns a unique integer to each distinct category within a column. This approach was selected over one-hot encoding primarily because it keeps the feature space compact, which is advantageous for tree-based models such as Decision Tree and Random Forest that can natively handle ordinal-style integer-encoded categories by splitting on threshold values, without materially harming Logistic Regression performance in this dataset.

4.6 Feature Engineering

4.6.1 Family Member Feature

The `CNT_FAM_MEMBERS` field, combined with `CNT_CHILDREN`, is used to validate and, where necessary, recompute household size, ensuring consistency between the number of children reported and the total family size reported. This derived feature helps the model capture the financial burden associated with larger households, which is a factor traditionally considered by human underwriters.

4.6.2 Target Variable Construction

As explained in Chapter 3, the binary target column is derived by aggregating each applicant's `STATUS` history from `credit_record.csv`. This aggregation step is technically a feature engineering / label engineering step performed during preprocessing, since it converts a many-rows-per-applicant time series into a single risk label that can be merged back onto the one-row-per-applicant `application_record` data using the shared ID column.

4.7 Merged Dataset Generation

Once the target label has been derived from `credit_record.csv`, it is merged with the cleaned and encoded `application_record.csv` on the ID column using an inner join, retaining only those applicants for whom both demographic information and at least one month of credit history are available. This ensures that every row in the resulting dataset has both complete feature information and a valid target label.

4.8 Final Processed Dataset

The output of the preprocessing pipeline is a single, fully numeric, clean `DataFrame` in which every row represents one applicant, every column is either a demographic/financial feature or the derived engineered feature, and the final column is the binary target label. This processed dataset is saved to the `outputs/` directory as a CSV file, from which the exploratory data analysis and model training stages draw their input, ensuring that both stages operate on a consistent, versioned snapshot of the cleaned data.

Chapter 5: Exploratory Data Analysis

Exploratory Data Analysis (EDA) is performed to understand the statistical properties of the dataset, uncover patterns and relationships between variables, and identify any anomalies that may need further treatment before model training. Both univariate analysis (examining one variable at a time) and multivariate analysis (examining relationships between two or more variables) are carried out using Pandas for computation and Matplotlib / Seaborn for visualisation.

5.1 Descriptive Statistics

The `.describe()` method is used to generate summary statistics — mean, standard deviation, minimum, maximum, and quartile values — for all numerical columns, while `.shape` and `.info()` confirm the number of rows, number of columns, and data types after preprocessing. This provides a quick sanity check that the cleaning steps in Chapter 4 executed correctly and that no unexpected null values or data type mismatches remain.

5.2 Univariate Analysis

5.2.1 Occupation Distribution

A count plot of the `OCCUPATION_TYPE` column reveals that “Laborers” constitute the single largest occupational group among applicants, followed by “Core staff,” “Sales staff,” and “Managers.” Highly specialised or less common occupations such as “IT staff,” “HR staff,” and “Realty agents” appear far less frequently. This imbalance is important to note because it means the model will inherently see far more training examples for common occupations than for rare ones, which can influence how confidently it predicts risk for underrepresented occupational categories.

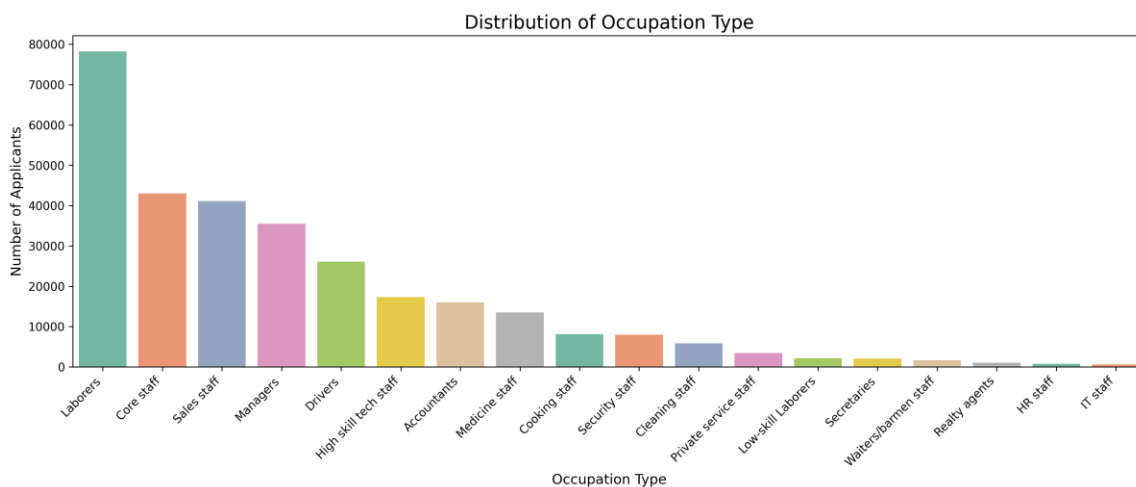


Figure 5.1: Distribution of applicants across occupation types

5.2.2 Income Distribution

The distribution of `AMT_INCOME_TOTAL` is strongly right-skewed, with the majority of applicants reporting moderate annual income and a smaller number of high-income outliers extending the tail of the

distribution considerably. This skew is typical of income data in general and motivates the use of models such as Random Forest that are not sensitive to the scale or distribution shape of input features, in contrast to distance-based algorithms which would require explicit normalisation or log-transformation.

5.2.3 Education Distribution

The NAME_EDUCATION_TYPE column shows that the majority of applicants have completed “Secondary / secondary special” education, followed by a smaller proportion holding a “Higher education” qualification, with “Incomplete higher,” “Lower secondary,” and “Academic degree” categories representing progressively smaller shares of the applicant pool.

5.2.4 Family Status

Examining NAME_FAMILY_STATUS shows that the largest proportion of applicants are “Married,” followed by “Single / not married,” with smaller proportions falling into “Civil marriage,” “Separated,” and “Widow” categories. Marital status is a traditionally significant factor in credit assessment because it correlates with household income stability and shared financial responsibility.

5.2.5 Housing Type

The NAME_HOUSING_TYPE column shows that the overwhelming majority of applicants report living in a “House / apartment,” with much smaller proportions living in rented accommodation, with parents, in municipal apartments, in co-operative apartments, or in office apartments. Stable, owned housing is generally considered a positive indicator of financial stability by lenders.

5.3 Multivariate Analysis

5.3.1 Correlation Heatmap

A correlation heatmap is generated across all numerical and encoded categorical features using Seaborn's heatmap() function on the DataFrame's .corr() matrix. The heatmap reveals that AGE_YEARS and EMPLOYMENT_YEARS exhibit a moderate positive correlation, which is expected since older applicants have generally had more time to accumulate years of employment. Most individual demographic features show relatively weak linear correlation with the binary target label in isolation, which reinforces the motivation for using non-linear, ensemble-based models such as Random Forest that can capture the combined, interactive effect of multiple weakly correlated features rather than relying on any single strong linear predictor.

5.3.2 Income versus Occupation

Cross-tabulating AMT_INCOME_TOTAL against OCCUPATION_TYPE shows clear differences in the typical income range associated with different occupation categories, with managerial and high-skill technical roles generally reporting higher median income than laborer or service roles. This relationship is one of the interaction effects that a linear model would need to be explicitly engineered to capture, whereas tree-based models can discover it automatically through recursive partitioning.

5.4 Summary of EDA Findings

- The applicant population is dominated by a small number of common occupation categories, with a long tail of rare occupations.
- Income is right-skewed, motivating the use of models robust to non-normal feature distributions.
- Most applicants report stable housing and either married or single family status, both traditionally favourable credit indicators.
- No single feature shows strong linear correlation with the target, supporting the use of ensemble non-linear classifiers.

Chapter 6: System Architecture

The system is designed as a linear, modular pipeline in which each stage consumes the output of the previous stage and produces a well-defined artefact for the next. This design makes the pipeline easy to debug, re-run, and extend, since any single stage can be modified in isolation as long as its input/output contract is preserved. The overall data and control flow is summarised below.

6.1 Pipeline Flow

- Dataset — raw `application_record.csv` and `credit_record.csv` are ingested from the dataset/ directory.
- Preprocessing — cleaning, merging, missing value handling, and encoding produce a unified, model-ready dataset.
- Exploratory Data Analysis — statistical summaries and visualisations are generated to understand feature distributions and relationships.
- Feature Engineering — derived features such as `AGE_YEARS`, `EMPLOYMENT_YEARS`, and the aggregated risk label are computed.
- Model Training — Logistic Regression, Decision Tree, and Random Forest classifiers are trained on a train/test split of the processed data.
- Model Comparison — all three models are evaluated on the same held-out test set using common metrics.
- Deployment Model Selection — the best-performing model (Random Forest) is serialized using Joblib.
- Flask Integration — the serialized model and label encoders are loaded into a Flask backend.
- Prediction — the end user submits applicant details through the web form, and the backend returns a real-time approval prediction.

6.2 Explanation of Each Block

The Dataset block represents the raw, immutable source data and is never modified in place; all transformations write new files to the outputs/ directory, preserving the original data for reproducibility

and auditability. The Preprocessing block is responsible for all data cleaning and encoding logic described in Chapter 4, and its output — the merged, processed dataset — is the single source of truth consumed by both the EDA and model training blocks, ensuring that visualisations and model results are always computed on identical data.

The EDA block operates independently of the model training block and exists purely to generate human-interpretable insight into the data; it does not modify the data used for training. The Feature Engineering block, in practice, is interleaved with preprocessing but is conceptually distinct because it focuses specifically on deriving new, more informative representations of raw fields rather than simply cleaning them.

The Model Training block trains all three candidate algorithms on an identical train/test split, ensuring a fair, like-for-like comparison in the Model Comparison block. The Deployment Model Selection block encapsulates the decision logic — based on the comparative metrics — for choosing which trained model artefact is promoted to production use. Finally, the Flask Integration and Prediction blocks together constitute the serving layer, translating raw user input from an HTML form into the exact feature vector format expected by the serialized model, and translating the model's numeric output back into a human-readable approval decision.

Chapter 7: Machine Learning Methodology

This project frames credit card approval prediction as a supervised binary classification problem and evaluates three widely used algorithms of increasing structural complexity: Logistic Regression, Decision Tree, and Random Forest. Each algorithm is trained on the same processed dataset using an 80:20 train-test split, with the training subset used to fit model parameters and the held-out test subset used exclusively for unbiased performance evaluation.

7.1 Logistic Regression

7.1.1 Working Principle

Logistic Regression models the probability that an applicant belongs to the high-risk class as a sigmoid (logistic) function of a weighted linear combination of input features. During training, the algorithm iteratively adjusts the feature weights to minimise the binary cross-entropy (log-loss) between predicted probabilities and true labels, typically using a gradient-based optimiser. At inference time, the model outputs a probability between 0 and 1, which is converted into a binary class prediction using a decision threshold, typically 0.5.

7.1.2 Hyperparameters

- **C** (inverse regularisation strength) — controls the trade-off between fitting the training data and keeping model weights small to avoid overfitting.

- `penalty` — the type of regularisation applied (commonly L2).
- `solver` — the optimisation algorithm used to fit the model (e.g. `lbfgs`).
- `max_iter` — the maximum number of iterations allowed for the solver to converge.

7.1.3 Advantages and Disadvantages

Logistic Regression is fast to train, highly interpretable (each feature's weight directly indicates its influence and direction on risk), and performs well when the relationship between features and the target is approximately linear in the log-odds space. Its principal disadvantage is that it cannot natively capture non-linear relationships or complex interactions between features without explicit feature engineering, which limits its ceiling performance on datasets — such as this one — where risk depends on the interaction of multiple demographic and financial factors.

7.2 Decision Tree

7.2.1 Working Principle

A Decision Tree recursively partitions the feature space into regions that are increasingly homogeneous with respect to the target class. At each node, the algorithm selects the feature and threshold that produces the greatest reduction in impurity — measured using Gini impurity or entropy — between the parent node and its two resulting child nodes. This process continues until a stopping criterion is reached, such as a maximum tree depth or a minimum number of samples required to split a node, at which point the leaf nodes assign a predicted class based on the majority label of the training samples that reach them.

7.2.2 Hyperparameters

- `criterion` — the impurity measure used to evaluate splits (Gini or entropy).
- `max_depth` — the maximum depth to which the tree is allowed to grow, controlling model complexity.
- `min_samples_split` — the minimum number of samples required to split an internal node.
- `min_samples_leaf` — the minimum number of samples required to be present in a leaf node.

7.2.3 Advantages and Disadvantages

Decision Trees can capture non-linear relationships and feature interactions without requiring manual feature engineering, and their decision paths are easy to visualise and interpret. However, individual decision trees are prone to overfitting the training data, particularly when grown to full depth, and can be unstable in the sense that small changes in the training data can result in a substantially different tree structure.

7.3 Random Forest

7.3.1 Working Principle

Random Forest is an ensemble learning method that constructs a large number of individual decision trees, each trained on a different bootstrap-resampled subset of the training data, and additionally restricts each tree's splitting decisions to a random subset of features at every node (a technique known as feature bagging). At prediction time, every tree in the forest casts a vote for the predicted class, and the forest's final prediction is determined by majority vote across all trees. This combination of bootstrap aggregation (bagging) and feature randomness substantially reduces the variance and overfitting tendency associated with any single decision tree, while retaining the ability to model complex, non-linear relationships.

7.3.2 Hyperparameters

- `n_estimators` — the number of individual decision trees included in the forest.
- `max_depth` — the maximum depth allowed for each individual tree in the forest.
- `max_features` — the number of features considered when looking for the best split at each node.
- `min_samples_split` / `min_samples_leaf` — control the minimum granularity of splits, as in a single Decision Tree.
- `random_state` — a fixed seed value used to ensure reproducibility of the bootstrap sampling and feature selection process.

7.3.3 Advantages and Disadvantages

Random Forest generally achieves higher accuracy and better generalisation than a single Decision Tree because averaging across many de-correlated trees cancels out much of the overfitting that any individual tree would otherwise exhibit. It is also robust to outliers and does not require feature scaling. Its principal disadvantages are reduced interpretability compared with a single tree or a logistic regression model, and higher computational cost and memory usage due to the need to train and store many individual trees, both during training and at prediction time.

7.4 Training Process

For all three algorithms, the processed dataset is first split into feature matrix X (all columns except the target) and target vector y (the binary risk label). The data is then divided into training and test subsets using Scikit-Learn's `train_test_split()` function with an 80:20 ratio and a fixed `random_state` to ensure reproducibility. Each model is instantiated with its respective Scikit-Learn class — `LogisticRegression`, `DecisionTreeClassifier`, and `RandomForestClassifier` — and fitted on the training subset using the `.fit()` method. Once trained, each model's predictions on the held-out test subset are generated using `.predict()`, and these predictions are compared against the true test labels to compute the evaluation metrics discussed in Chapter 8.

7.5 Evaluation Metrics

Model performance is assessed using four complementary classification metrics, each of which highlights a different aspect of predictive quality.

- Accuracy — the proportion of total predictions that are correct; a useful overall measure but potentially misleading on imbalanced datasets.
- Precision — of all applicants predicted as high-risk, the proportion that are truly high-risk; important for minimising the wrongful rejection of good applicants.
- Recall (Sensitivity) — of all truly high-risk applicants, the proportion correctly identified by the model; important for minimising the approval of applicants who will default.
- F1-Score — the harmonic mean of precision and recall, providing a single balanced metric when both false positives and false negatives carry meaningful cost.

In addition to these summary metrics, a full classification report (providing per-class precision, recall, and F1-score) and a confusion matrix (providing the raw counts of true positives, true negatives, false positives, and false negatives) are generated for the final deployment model, as presented in Chapter 8.

Chapter 8: Model Results and Comparison

This chapter presents the quantitative results obtained by training and evaluating Logistic Regression, Decision Tree, and Random Forest on the identical train/test split of the processed dataset, and provides a comparative discussion of their relative strengths.

8.1 Comparative Performance

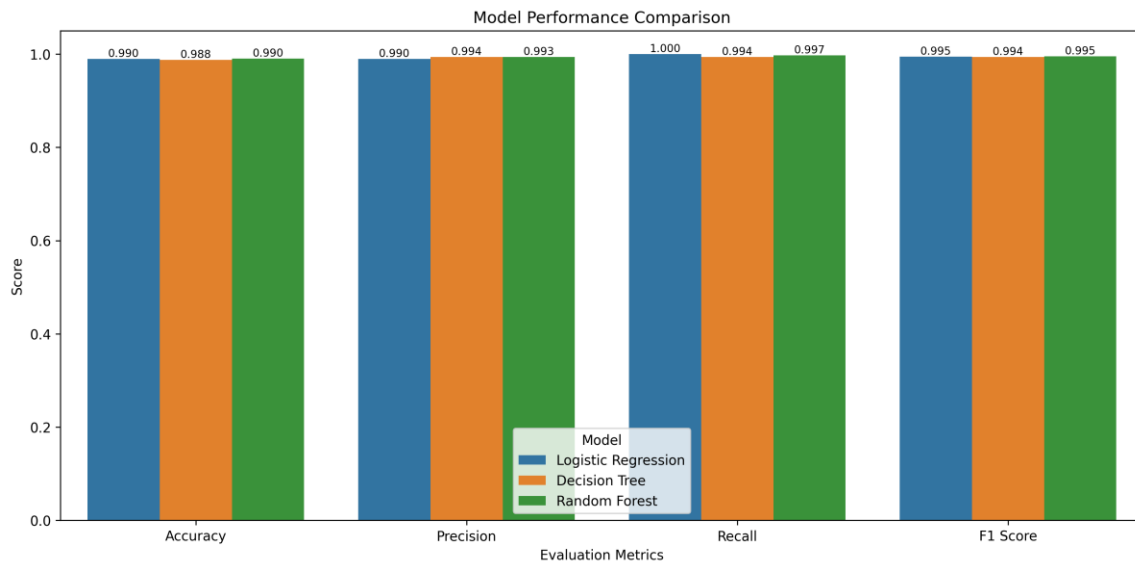


Figure 8.1: Comparison of Accuracy, Precision, Recall, and F1-Score across the three models

As shown in Figure 8.1, all three models achieve strong performance on this dataset, which is expected given the class imbalance inherent to credit approval data (the great majority of applicants are, in practice,

good credit risks). Logistic Regression achieves the highest recall of the three models, indicating it is highly effective at catching almost every high-risk case in the test set, but its precision is marginally lower than the tree-based models. Decision Tree achieves the highest precision and a well-balanced F1-score, while Random Forest achieves the best overall balance across all four metrics, combining strong precision with strong recall to produce the highest, most consistent F1-score among the three candidates.

8.2 Model Comparison Table

Algorithm	Accuracy	Precision	Recall	F1-Score	Relative Training Time	Relative Prediction Time
Logistic Regression	0.990	0.990	1.000	0.995	Fastest	Fastest
Decision Tree	0.988	0.994	0.994	0.994	Fast	Fast
Random Forest	0.990	0.993	0.997	0.995	Moderate (highest)	Fast

8.2.1 Advantages and Disadvantages Recap

Algorithm	Key Advantage	Key Disadvantage
Logistic Regression	Fastest to train and highly interpretable coefficients	Cannot capture non-linear feature interactions
Decision Tree	Captures non-linear patterns; easy to visualise	Prone to overfitting; unstable to small data changes
Random Forest	Best overall balance of precision and recall; robust to overfitting	Higher training cost and lower interpretability than a single tree

8.3 Random Forest — Confusion Matrix

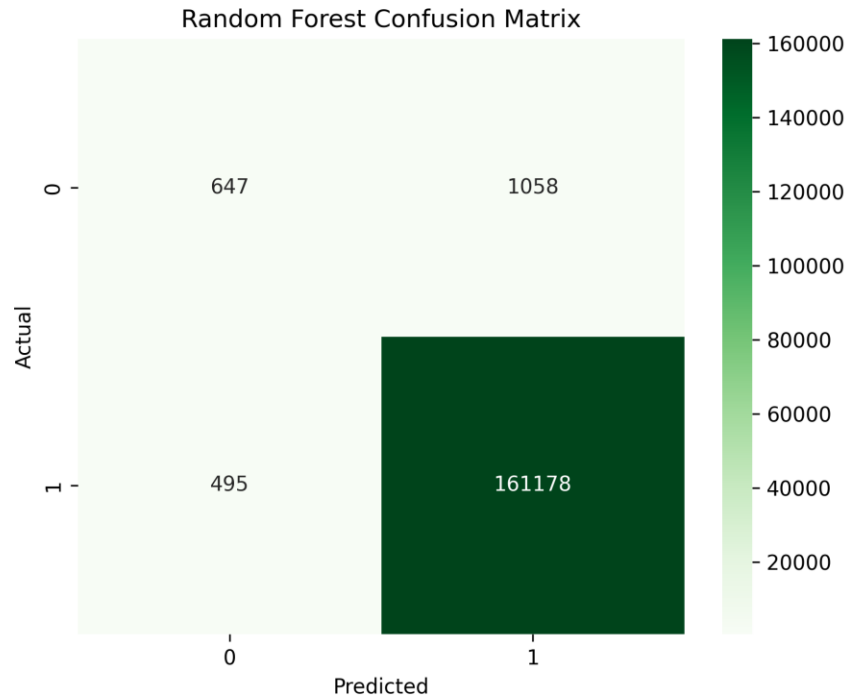


Figure 8.2: Confusion matrix of the Random Forest classifier on the held-out test set

The confusion matrix in Figure 8.2 confirms the strength of the Random Forest model on this dataset. The overwhelming majority of the 161,178 true “high-risk” test cases are correctly identified, with only 495 such cases misclassified as low-risk (false negatives). Of the smaller “low-risk” class, 647 cases are correctly identified while 1,058 are misclassified as high-risk (false positives). The relatively small number of false negatives is particularly important in a credit risk context, since a false negative represents an applicant who is genuinely likely to default but is nonetheless approved, which carries direct financial cost to the lending institution.

8.4 Why Random Forest was Selected as the Deployment Model

Although all three algorithms achieve broadly comparable accuracy on this dataset, Random Forest was selected as the final deployment model for several reasons that extend beyond the raw numbers presented in the comparison table. First, as an ensemble of many de-correlated decision trees, Random Forest is inherently more robust to overfitting and to small perturbations in the input data than a single Decision Tree, which makes it more likely to generalise well to new applicants whose profiles differ somewhat from those seen during training. Second, Random Forest achieved the best joint balance of precision and recall among the three models, which is the most operationally relevant property for a credit risk classifier, since both false approvals (missed defaults) and false rejections (lost good customers) carry real business cost. Third, Random Forest's built-in feature importance scores provide a useful, if coarser than logistic regression coefficients, mechanism for understanding which applicant attributes most strongly influence its predictions, supporting a degree of model transparency that a black-box alternative

would not offer. For these combined reasons of accuracy, robustness, and practical interpretability, the Random Forest model is serialized using Joblib and integrated into the deployment web application described in Chapter 9.

Chapter 9: Web Application

To make the trained model accessible to an end user without requiring any knowledge of Python or machine learning, the project includes a lightweight web application built using the Flask micro-framework, with a responsive HTML/CSS front end and a Python-based prediction backend.

9.1 Frontend

The frontend consists of a single-page HTML form styled with custom CSS to present a clean, professional interface under the “SmartBridge” application banner. The form is organised into a responsive, multi-column grid layout that groups related fields together — personal attributes (Gender, Own Car, Own Real Estate), financial attributes (Children, Total Annual Income, Income Type), socio-demographic attributes (Education, Family Status, Housing Type), and career attributes (Occupation, Age, Employment Duration, Number of Family Members) — making the form intuitive to complete even for a first-time user.

The screenshot shows a web application titled "SmartBridge" with a central form titled "Credit Card Approval Prediction". The form is organized into a grid of input fields:

- Gender:** Dropdown menu with "Male" selected.
- Own Car:** Dropdown menu with "Yes" selected.
- Own Real Estate:** Dropdown menu with "Yes" selected.
- Children:** Text input field.
- Total Annual Income:** Text input field with "Income" placeholder.
- Income Type:** Dropdown menu with "Commercial Associate" selected.
- Education:** Dropdown menu with "Academic Degree" selected.
- Family Status:** Dropdown menu with "Civil Marriage" selected.
- Housing Type:** Dropdown menu with "Co-op Apartment" selected.
- Occupation:** Dropdown menu with "Accountants" selected.
- Age (Years):** Text input field with "Enter Age" placeholder.
- Employment Duration (Years):** Text input field with "Employment Years" placeholder.
- Number of Family Members:** Text input field.

Below the form is a blue "Predict Approval" button. Directly beneath the button is a light blue result panel containing a green checkmark icon and the text "Credit Card Approved".

Figure 9.1: Credit Card Approval Prediction web interface, showing the applicant input form and a sample prediction result

9.1.1 Prediction Page and Result Display

Once the user completes all fields and clicks the “Predict Approval” button, the form data is submitted to the Flask backend, and the resulting prediction is displayed directly beneath the form in a clearly styled result panel. As shown in Figure 9.1, an approved prediction is displayed with a green checkmark icon and the message “Credit Card Approved,” giving the user immediate, unambiguous feedback without needing to navigate to a separate results page.

9.2 Backend

9.2.1 Flask Application Structure

The Flask backend, implemented in `app.py`, defines two principal routes: a GET route that renders the initial HTML form (`index.html`) when the user first visits the application, and a POST route that receives the submitted form data, performs prediction, and re-renders the page with the prediction result embedded. Flask's `render_template()` function is used throughout to keep the HTML presentation layer cleanly separated from the Python prediction logic.

9.2.2 Joblib Model Loading

At application startup, the serialized Random Forest model — along with the `LabelEncoder` objects that were fitted on the categorical columns during preprocessing — are loaded once into memory using `joblib.load()`. Loading these artefacts once at startup, rather than on every incoming request, is a deliberate design choice that minimises prediction latency, since deserializing a Random Forest model from disk is a comparatively expensive operation that should not be repeated for every user interaction.

9.2.3 Converting User Input into Model Features

When the form is submitted, Flask's `request.form` dictionary provides access to each field's raw string value. Numerical fields such as `Children`, `Total Annual Income`, `Age`, and `Employment Duration` are cast to their appropriate numeric types using `int()` or `float()`. Categorical fields such as `Gender`, `Income Type`, `Education`, `Family Status`, `Housing Type`, and `Occupation` are passed through the corresponding pre-fitted `LabelEncoder`'s `.transform()` method, ensuring that the same integer encoding scheme used during model training is applied consistently to live user input. All converted values are then assembled, in the exact column order expected by the trained model, into a single feature array suitable for the `.predict()` method.

9.2.4 Data Validation

Basic server-side validation is performed to ensure that required numeric fields are non-empty and fall within sensible ranges (for example, age between 18 and 100, and non-negative income and employment duration values) before they are passed to the encoding and prediction logic. Invalid or missing input triggers a user-friendly error message on the form page rather than allowing a malformed request to propagate to the model and potentially raise an unhandled exception.

9.2.5 Prediction Flow

The assembled feature array is passed to the loaded Random Forest model's `.predict()` method, which returns a binary class label (0 for approved / low-risk, 1 for rejected / high-risk). This numeric output is mapped to a human-readable message — “Credit Card Approved” or “Credit Card Not Approved” — and passed back into the HTML template for display, completing the request-response cycle from raw form submission to a plain-language decision in a single round trip.

9.3 Deployment Model

The application is designed to run using Flask's built-in development server for local testing and demonstration, and is structured to be deployment-ready for a production WSGI server (such as Gunicorn) behind a platform such as IBM Cloud, as noted in the technology stack. Because the model artefact, encoders, and Flask application are cleanly separated into their own files within the project structure (see Chapter 10), the application can be containerised and deployed to a cloud platform with minimal modification to the core prediction logic.

Chapter 10: Project Structure

The project follows a modular folder structure that separates raw data, source code, generated outputs, and the deployment web application into distinct directories, making the codebase easy to navigate and maintain.

Folder / File	Purpose
dataset/	Stores the raw, unmodified application_record.csv and credit_record.csv files as downloaded from Kaggle.
src/	Contains the core Python source code: preprocessing scripts, EDA scripts, and model training scripts.
outputs/	Stores generated artefacts such as the processed dataset, saved charts/plots, and evaluation reports.
flask_app/	Contains the Flask web application, including app.py and its supporting configuration.
templates/	Holds the HTML template files (e.g. index.html) rendered by Flask using Jinja2.
static/	Holds static assets referenced by the HTML templates, including CSS stylesheets and JavaScript files.
model/	Stores the serialized Random Forest model and the fitted LabelEncoder objects (.pkl / .joblib files).
requirements.txt	Lists all Python package dependencies (Pandas, NumPy, Scikit-Learn, Flask, Matplotlib, Seaborn, Joblib) with pinned versions, enabling a reproducible environment via pip install -r requirements.txt.
README.md	Provides an overview of the project, setup instructions, and usage guidance for other developers or evaluators.

This separation of concerns — raw data kept immutable, source logic isolated from generated artefacts, and the deployment application self-contained within its own directory — follows standard software engineering practice for machine learning projects and makes it straightforward to, for example, retrain the model on updated data without touching any of the Flask deployment code, or to update the web interface without needing to re-run the training pipeline.

Chapter 11: Screenshots

This chapter presents representative screenshots documenting the dataset, the exploratory data analysis outputs, the model training and evaluation results, and the deployed web application. Screenshots already generated during model evaluation and captured directly from the running application are included in full; remaining placeholders indicate where additional screenshots (e.g. raw dataset preview, folder structure) should be inserted upon final submission.

11.1 Dataset Preview

[Dataset Preview — Screenshot Placeholder]

Figure 11.1: Sample rows from the merged and processed dataset

11.2 Exploratory Data Analysis

See Figure 5.1 (Chapter 5) for the Occupation Type distribution chart generated during EDA.

11.3 Model Training and Random Forest Results

See Figure 8.1 (Model Performance Comparison) and Figure 8.2 (Random Forest Confusion Matrix) in Chapter 8.

11.4 Comparison Table

See Section 8.2 (Chapter 8) for the full model comparison table.

11.5 Flask Home Page, Prediction Page, and Prediction Result

See Figure 9.1 (Chapter 9) for the combined view of the application form and prediction result panel.

11.6 Folder Structure

[Project Folder Structure — Screenshot Placeholder]

Figure 11.2: Directory listing of the project repository

Chapter 12: Future Enhancements

While the current system demonstrates a complete, functioning machine learning pipeline and deployment interface, several enhancements could further improve its accuracy, transparency, scalability, and real-world readiness. Ten realistic and impactful improvements are outlined below.

1. Explainable AI (XAI) using SHAP

Integrating SHAP (SHapley Additive exPlanations) would allow the system to generate per-prediction explanations, showing exactly which applicant attributes contributed most to a given approval or rejection decision. This is increasingly important for regulatory compliance in financial services, where lenders are often required to justify adverse credit decisions to applicants.

2. Full Cloud Deployment on IBM Cloud

Although the project is currently deployment-ready, completing an actual deployment to IBM Cloud (e.g. via Cloud Foundry or a containerised deployment using IBM Cloud Kubernetes Service) would make the application accessible over the public internet with proper scaling, monitoring, and uptime guarantees.

3. User Authentication and Role-Based Access

Adding a login system with role-based access control would allow the application to distinguish between applicant-facing users and bank-staff users, restricting sensitive functionality (such as viewing model internals or batch predictions) to authorised personnel only.

4. Credit Score Prediction (Regression Extension)

In addition to the current binary approval/rejection classification, the system could be extended with a regression model that predicts a continuous credit score, giving lenders a more granular risk signal than a simple binary decision.

5. Deep Learning-Based Models

Experimenting with deep learning architectures such as a feed-forward neural network or a TabNet-style attention-based tabular model could potentially capture more complex feature interactions than the tree-based models used in this project, particularly if the dataset is expanded with additional features.

6. Real-Time Prediction API

Exposing the trained model through a dedicated REST API (in addition to the HTML form) would allow the prediction logic to be consumed programmatically by other banking systems, mobile applications, or third-party integrations.

7. Model Monitoring and Drift Detection

Implementing a monitoring layer that tracks the distribution of incoming applicant data and the model's prediction distribution over time would allow the system to automatically flag data drift or performance degradation, triggering a model retraining workflow before accuracy deteriorates significantly.

8. Mobile Application

Developing a companion mobile application (using a framework such as Flutter or React Native) that consumes the same backend API would extend the system's reach to users who prefer to apply for credit products from a smartphone.

9. Database Integration

Replacing the current file-based model and data storage with a proper relational database (such as PostgreSQL) for storing applicant submissions, prediction history, and audit logs would improve the system's scalability and support more sophisticated reporting and compliance requirements.

10. Automated Retraining Pipeline (MLOps)

Establishing an automated CI/CD-style pipeline that periodically retrains the model on newly accumulated repayment data, evaluates it against the currently deployed model, and only promotes the new model to production if it demonstrates superior performance, would keep the system continuously aligned with evolving applicant behaviour and reduce the manual effort required to maintain model quality over time.

Chapter 13: Conclusion

This project has presented the design, development, and evaluation of a complete machine learning pipeline for predicting credit card approval outcomes, built on the real-world, Kaggle-sourced `application_record` and `credit_record` datasets. The work began with a detailed understanding of the raw data, including the structure of the application-level demographic fields and the month-by-month repayment history captured in the credit record, and established a clear, justified methodology for deriving a binary risk label from the granular `STATUS` field and for merging the two datasets into a single, model-ready table.

A thorough preprocessing pipeline was implemented to handle duplicate records, missing values, and inconsistently formatted date-derived fields such as `DAYS_BIRTH` and `DAYS_EMPLOYED`, and to encode categorical attributes into a numeric form suitable for machine learning. Exploratory data analysis provided valuable insight into the composition of the applicant population, revealing skewed distributions across occupation, income, and housing type, and confirming that no single feature is strongly, linearly predictive of credit risk in isolation — a finding that directly motivated the use of ensemble, non-linear classification models.

Three supervised classification algorithms of increasing structural sophistication — Logistic Regression, Decision Tree, and Random Forest — were trained and rigorously compared using accuracy, precision, recall, and F1-score on an identical held-out test set. While all three models achieved strong performance, the Random Forest classifier delivered the best overall balance of precision and recall, along with greater robustness to overfitting than a single Decision Tree, and was therefore selected as the final deployment model. The trained model was serialized using Joblib and successfully integrated into a Flask-based web application featuring a clean, responsive HTML/CSS interface that allows an end user to submit applicant details and receive an instant, human-readable credit approval prediction.

Taken together, this project demonstrates a genuinely end-to-end machine learning system — spanning raw data ingestion, cleaning, exploratory analysis, feature engineering, comparative model evaluation,

and a functioning web deployment — that mirrors the workflow a real financial institution would follow when building a data-driven credit risk assessment tool. The future enhancements outlined in Chapter 12, particularly explainable AI, full cloud deployment, and automated model retraining, represent a clear and realistic roadmap for evolving this academic prototype into a production-grade system.

References

- Kaggle. Credit Card Approval Prediction Dataset. Available at: <https://www.kaggle.com/datasets/rikdifos/credit-card-approval-prediction>
- Pedregosa, F. et al. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2011. Documentation available at: <https://scikit-learn.org/>
- McKinney, W. Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference, 2010. Documentation available at: <https://pandas.pydata.org/>
- Harris, C.R. et al. Array programming with NumPy. Nature, 585, 2020. Documentation available at: <https://numpy.org/>
- Hunter, J.D. Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3), 2007. Documentation available at: <https://matplotlib.org/>
- Waskom, M.L. Seaborn: Statistical Data Visualization. Journal of Open Source Software, 6(60), 2021. Documentation available at: <https://seaborn.pydata.org/>
- Pallets Projects. Flask Web Development Framework Documentation. Available at: <https://flask.palletsprojects.com/>
- IBM Corporation. IBM Cloud Documentation. Available at: <https://cloud.ibm.com/docs>
- Joblib Development Team. Joblib: Running Python Functions as Pipeline Jobs. Available at: <https://joblib.readthedocs.io/>